

WERGONIC · QUALITY

How We Do QA

A plain-language guide to how we test our products and catch problems before our users do — across the website apps, the backend, and the mobile app.

Audience: Quality (QA) team & new hires

Products: backend · web apps · mobile app

Purpose: shared understanding of our testing approach

1 · The goal, in one line

Find problems ourselves — quickly and automatically — before a real user ever runs into them.

Every product has bugs. The question is **who finds them first**: us, or the customer. Our whole approach is built so that the moment a developer changes the code, a set of automatic checks runs and tells us if something broke. If it did, that change is blocked from going live until it's fixed.

A quick vocabulary so the rest of this document is easy to read:

Word	What it means (plain)
A test	A small piece of code (or a checklist for a human) that confirms one thing works the way it should.
Automatic checks	A system connected to our code that runs all our tests by itself every time code changes. (The technical name is "CI/CD" — you'll hear it, but you can just think "the automatic checker".)
Manual testing	A human using the app by hand to confirm it looks and behaves correctly.
Bug ticket	A written record of a problem you found, so the team knows what to fix.
Staging / Production	"Staging" is a private copy of the app for testing. "Production" is the real live app that customers use.

Where we are today (honest starting point). Right now we have very few automatic tests, so most problems are only found by chance or by users. Building this testing system — and having QA cover the rest by hand — is exactly the job we're setting up.

2 · The simple rule: many small tests, few big ones

Small tests are fast and cheap, so we write lots of them. Big tests are slow and fragile, so we keep them for the few most important journeys.

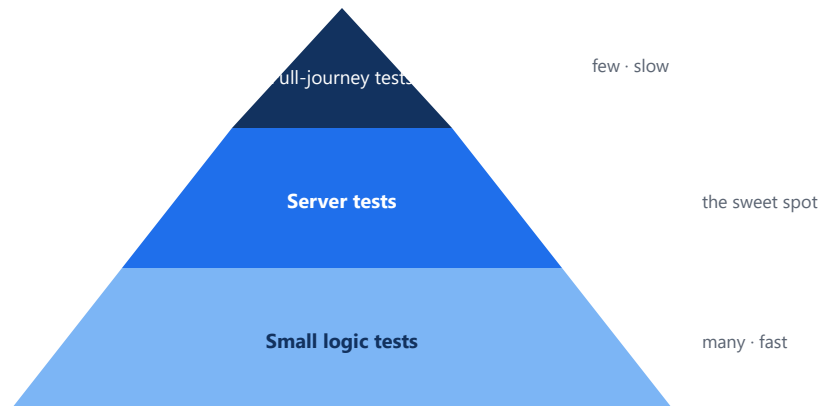


Figure 1 — Fewer tests as you go up; each one is bigger and slower.

There are really only four kinds of testing you need to know. Here they are, from smallest to biggest, with a real example from our products:

Type	What it checks	Example from us
Small logic test	One small piece of logic on its own — usually a calculation.	"Given these sensor angles, does the posture score come out correct?"
Server test	Does the backend answer the app correctly and save the right data?	"When the app asks to create a work session, does the server save it and send it back?"
Full-journey test	Acts like a real user clicking through several screens.	"Log in → start a session → open the report — does the whole flow work?"
Manual test	A human checks things a machine can't easily judge — look, feel, and real hardware.	"Does the report page look right, and does the real sensor connect?"

Why the middle matters most. The "server tests" are our best investment because all three products — the two websites and the mobile app — depend on the same backend. One bug there can break everything at once, so that's where we test the hardest.

3 · When do the tests run?

Not all at once. Fast checks run constantly; slow checks run less often. This is how we get quick answers without waiting forever.

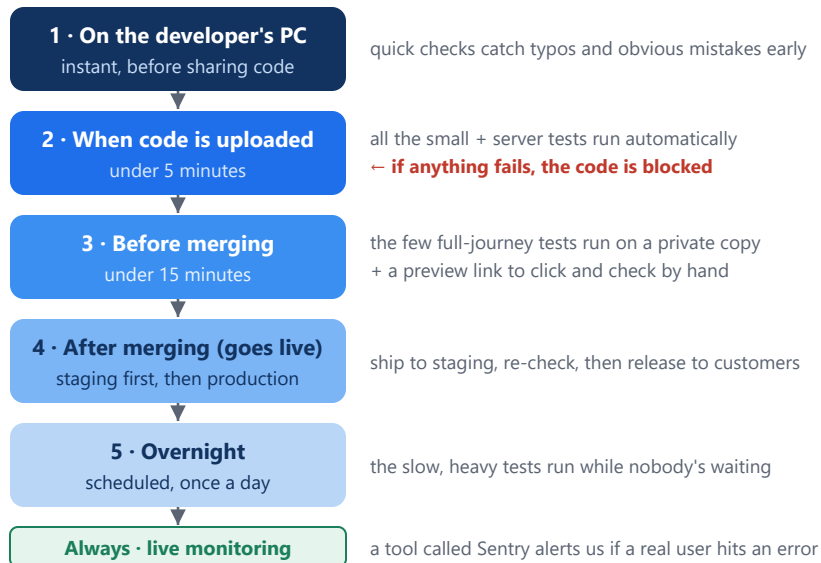


Figure 2 — The same tests, run at different moments, from fastest to always-on.

4 · What we test in each product

Same idea everywhere, adapted to each product.

Product	Main focus	Tools used
Backend the shared server	Server tests on the important actions (create a session, register a company, manage users), plus small tests on the calculations.	pytest
Web apps client panel + admin	Small tests on forms and logic, plus a handful of full-journey tests for the main flows.	Vitest, Playwright
Mobile app phone + sensors	Small tests on the data calculations, some automated flows on a phone simulator, and hands-on checks with a real device.	Flutter test, manual

Alongside all of this, QA does **manual testing** — using each product by hand, feature by feature, and writing a bug ticket for anything that's wrong. Manual testing and automatic tests are partners: the automatic tests guard against old bugs coming back, and the manual testing finds the new ones.

5 · The mobile sensors — the tricky part, made simple

The phone app talks to real body-worn motion sensors over Bluetooth. You can't fully automate real hardware — but you can automate far more of it than people expect.

The trick: the *hard, bug-prone part* isn't the Bluetooth connection itself — it's what the app *does with the sensor data* afterwards (the calculations, the posture scores, saving and syncing). That part is just code, and code can be tested automatically. So we split sensor testing into three levels:

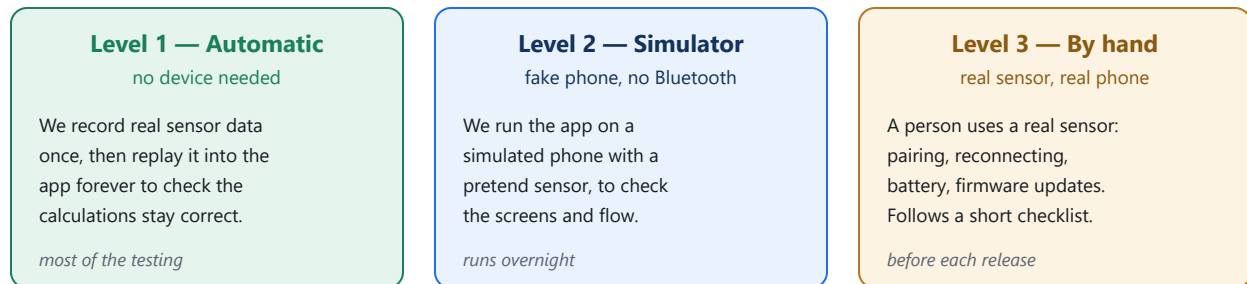


Figure 3 — Automate the data (Levels 1–2); test the physical connection by hand (Level 3).

So sensors are **not** "all manual". Only the physical Bluetooth connection really needs a person with a real device, and that becomes a short, repeatable checklist done before each mobile release — not guesswork.

6 · How we build this, step by step

We don't do everything at once. Each step adds real protection before the next begins.

Step	What we do	Why it's first / next
1	Turn on the "block bad code" rule and set up the automatic checker properly.	Right now a broken change can still go live. This stops that.
2	Write the first real tests for the backend (the shared server).	Biggest payoff — it protects all three products at once.
3	Add tests for the main website flows and forms.	Covers what customers touch most.
4	Set up the mobile tests, including the sensor-data replay.	Makes the hardest product testable without hardware.
5	Add preview links, staging checks, and live monitoring.	Confidence before and after each release.

The whole idea in one sentence. Many quick checks on every change, strong tests on the shared server, a few full user-journey tests, sensor calculations tested by replaying recordings, and a live alarm (Sentry) if a real user hits an error.